

LAPORAN TUGAS BESAR
MIKROPROSESOR DAN ANTARMUKA
SMART RAIN-ACTIVATED SWITCH SYSTEM
IMPLEMENTASI ADC DENGAN RAIN SENSOR FC-37, SERVO, DAN
I2C DISPLAY BERBASIS CMSIS UNTUK HOME AUTOMATION



Kelompok : 7

Ridho Anugrah Mulyadi 101032300028

Muhammad Hafidz Abdurrahman 101032300151

PROGRAM STUDI S1 TEKNIK KOMPUTER
FAKULTAS TEKNIK ELEKTRO
UNIVERSITAS TELKOM
2025

DAFTAR ISI

DAFTAR ISI.....	2
BAB 1 PENDAHULUAN	3
1.1. Latar Belakang	3
1.2. Rumusan Masalah.....	3
1.3. Tujuan	3
1.4. Batasan Masalah	3
1.5. Manfaat	3
BAB II PERANCANGAN SISTEM.....	5
2.1. Arsitektur Sistem	5
2.1.1. Schematic Diagram.....	5
2.2. Desain Hardware.....	5
2.2.1. Schematic Diagram.....	5
2.3. Desain Software	6
2.3.1. Flowchart Diagram	6
2.3.2. State Machine Diagram.....	6
BAB III DOKUMENTASI	7
3.1 Source Code	7
3.2 Demonstrasi Alat	14
3.3 Realisasi Alat	14
BAB IV KESIMPULAN DAN SARAN	17
6.1 Kesimpulan	17
6.2 Saran	17
DAFTAR PUSTAKA.....	18

BAB 1 PENDAHULUAN

1.1. Latar Belakang

Perubahan cuaca yang tiba-tiba seringkali menuntut respons cepat untuk mengamankan aset atau memberikan peringatan. Dalam konteks home automation, deteksi hujan otomatis dapat digunakan untuk menutup jemuran, menutup jendela, atau menyalakan lampu peringatan. Proyek ini memanfaatkan sensor hujan tipe FC-37 yang memiliki sensitivitas tinggi dan harga terjangkau.

Mikrokontroler STM32F401CCU6 dipilih karena memiliki peripheral ADC 12-bit yang presisi untuk membaca data analog dari FC-37, serta Timer canggih untuk membangkitkan sinyal PWM bagi motor servo. Selain itu, penggunaan protokol komunikasi I2C untuk LCD memungkinkan efisiensi penggunaan pin GPIO. Pendekatan pemrograman berbasis register diterapkan untuk memahami arsitektur ARM Cortex-M3 secara mendalam, menghindari abstraksi library yang seringkali menyembunyikan detail operasional hardware.

1.2. Rumusan Masalah

1. Bagaimana mengkonfigurasi register ADC pada STM32 untuk membaca sinyal analog dari modul sensor FC-37?
2. Bagaimana menghasilkan sinyal PWM 50Hz yang stabil menggunakan Timer Register untuk mengendalikan posisi sudut servo?
3. Bagaimana mengintegrasikan pembacaan sensor dan aktuator servo dengan tampilan status pada LCD via protokol I2C berbasis register?
4. Bagaimana menentukan nilai threshold ADC yang optimal untuk membedakan kondisi gerimis dan hujan deras?

1.3. Tujuan

Adapun tujuan dari dibuatnya rancang bangun ini adalah:

1. Merancang sistem otomatis yang responsif terhadap kondisi hujan.

1.4. Batasan Masalah

1. Menggunakan hardware terpilih STM32F401CCU6 Blackpill, Sensor Hujan FC-37 + Modul LM393, Servo SG90, LCD 16x2 I2C.
2. Menggunakan Embedded C dan pemrograman berbasis Register (CMSIS)
3. Mekanisme teknis, servo sebagai aktuator akan menutup jendela atau menepikan jemuran

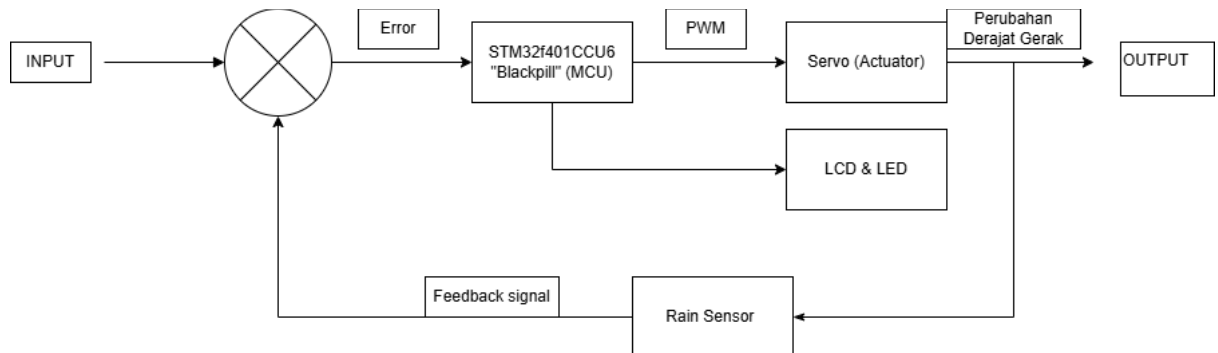
1.5. Manfaat

1. Manfaat Pembelajaran: Memberikan pemahaman mendalam mengenai arsitektur Low-Level mikrokontroler STM32F401CCU6, khususnya dalam manajemen register

BAB II PERANCANGAN SISTEM

2.1. Arsitektur Sistem

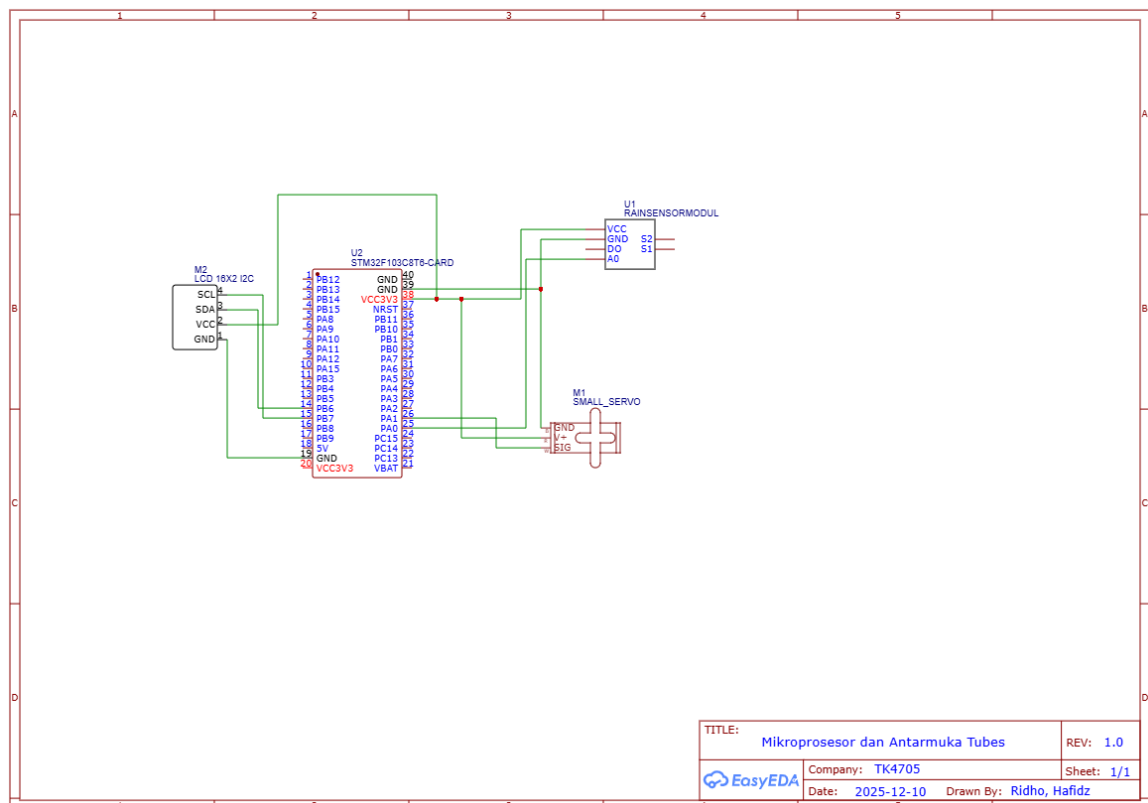
2.1.1. Schematic Diagram



Gambar 1 Block Diagram

2.2. Desain Hardware

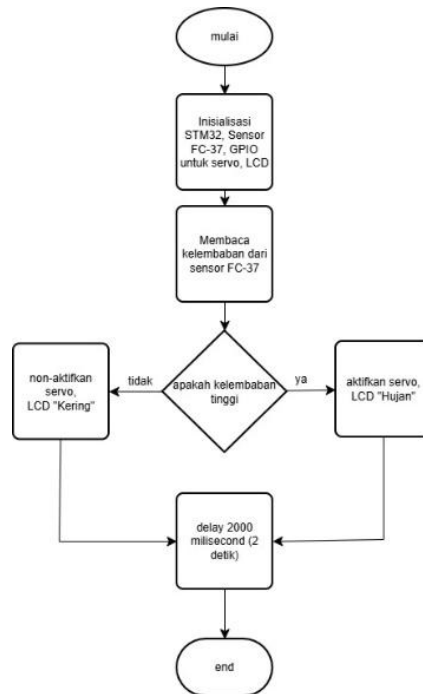
2.2.1. Schematic Diagram



Gambar 2 Schematic Diagram

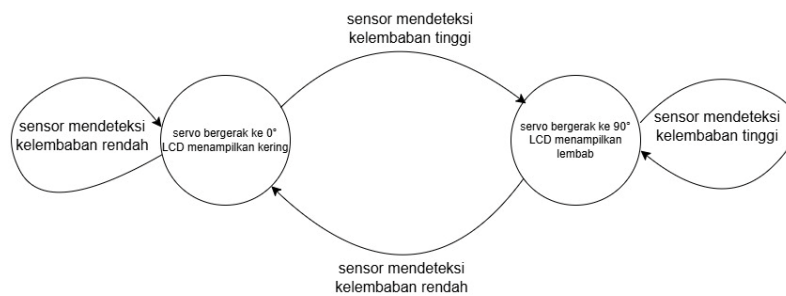
2.3. Desain Software

2.3.1. Flowchart Diagram



Gambar 3 Flowchart

2.3.2. State Machine Diagram



Gambar 4 State Machine Diagram

BAB III DOKUMENTASI

3.1 Source Code

```
#include "stm32f4xx.h"

/* konfigurasi threshold dan alamat LCD */
#define RAIN_THRESHOLD 2000
#define SERVO_OPEN 1000 // 0 Derajat
#define SERVO_CLOSE 2000 // 90 Derajat
#define LCD_ADDR 0x27 // Alamat umum LCD I2C (bisa juga 0x3F)

/* variabel global */
volatile uint32_t sensor_value = 0;
uint8_t current_state = 0; // 0 = Unknown, 1 = Hujan, 2 = Kering

/* fungsi */
void SystemClock_Config(void);
void GPIO_Init(void);
void ADC_Init(void);
void TIM2_Init(void);
void I2C1_Init(void); // Setup I2C1
void LCD_Init(void); // Setup LCD
void LCD_String(char *str); // Print String
void LCD_Clear(void); // Hapus Layar
void LCD_SetCursor(uint8_t row, uint8_t col);
void delay_ms(uint32_t ms);

// Fungsi Bantuan I2C/LCD Internal
void I2C1_Write(uint8_t data);
void LCD_SendCmd(uint8_t cmd);
void LCD_SendData(uint8_t data);

int main(void)
{
    // 1. Setup Clock
    SystemClock_Config();

    // 2. Inisialisasi Hardware
    GPIO_Init();
    ADC_Init();
    TIM2_Init();
    I2C1_Init(); // Init I2C untuk LCD

    // 3. Init LCD
```

```

delay_ms(50); // Tunggu power LCD stabil
LCD_Init();

// Tampilan Awal
LCD_Clear();
LCD_SetCursor(0, 0);
LCD_String("SYSTEM READY");
delay_ms(1000);
LCD_Clear();

while (1)
{
    // A. BACA SENSOR (ADC)
    ADC1->CR2 |= ADC_CR2_SWSTART;
    while (!(ADC1->SR & ADC_SR_EOC))
        ;
    sensor_value = ADC1->DR;

    // B. LOGIKA untuk LCD
    // (Hanya update tulisan jika status berubah)

    if (sensor_value < RAIN_THRESHOLD)
    {
        // KONDISI: HUJAN
        TIM2->CCR2 = SERVO_CLOSE;
        GPIOC->BSRR = (1 << 29); // LED ON (Reset PC13)

        if (current_state != 1)
        { // Jika status sebelumnya bukan hujan
            LCD_Clear();
            LCD_SetCursor(0, 2); // Tengah baris 1
            LCD_String("STATUS:");
            LCD_SetCursor(1, 4); // Tengah baris 2
            LCD_String("HUJAN!"); // Tampilkan
            current_state = 1;
        }
    }
    else
    {
        // KONDISI: KERING
        TIM2->CCR2 = SERVO_OPEN;
        GPIOC->BSRR = (1 << 13); // LED OFF (Set PC13)

        if (current_state != 2)

```



```

    { // Jika status sebelumnya bukan kering
      LCD_Clear();
      LCD_SetCursor(0, 2);
      LCD_String("STATUS:");
      LCD_SetCursor(1, 3);
      LCD_String("AMAN");
      current_state = 2;
    }
  }

  delay_ms(100);
}

/* I2C dan LCD DRIVER */
void I2C1_Init(void)
{
  // 1. Enable Clock GPIOB (untuk PB6 & PB7) dan I2C1
  RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
  RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;

  // 2. Setup PB6 (SCL) & PB7 (SDA) ke Alternate Function
  GPIOB->MODER &= ~(3 << (6 * 2)) | (3 << (7 * 2)); // Clear
  GPIOB->MODER |= ((2 << (6 * 2)) | (2 << (7 * 2))); // AF Mode
  GPIOB->OTYPER |= ((1 << 6) | (1 << 7)); // Open Drain
  GPIOB->OSPEEDR |= ((3 << (6 * 2)) | (3 << (7 * 2))); // High Speed
  GPIOB->PUPDR |= ((1 << (6 * 2)) | (1 << (7 * 2))); // Pull Up

  // Set AF4 (I2C1) pada AFR low register (AFRL)
  GPIOB->AFR[0] &= ~(0xF << (6 * 4)) | (0xF << (7 * 4));
  GPIOB->AFR[0] |= ((0x4 << (6 * 4)) | (0x4 << (7 * 4)));

  // 3. Konfigurasi I2C1
  I2C1->CR1 |= I2C_CR1_SWRST; // Reset I2C
  I2C1->CR1 &= ~I2C_CR1_SWRST;

  // Frekuensi APB1 = 42MHz. Set bit FREQ di CR2
  I2C1->CR2 |= 42;

  // Konfigurasi Clock Control (CCR) untuk 100kHz Standard Mode
  // Thigh = Tlow = CCR * TPCLK1
  // CCR = 42MHz / (2 * 100kHz) = 210
  I2C1->CCR = 210;

```

```

// TRISE (Max Rise Time). Untuk 100kHz => (1000ns / TPCLK1) + 1
// (1000ns / 23.8ns) + 1 = 43
I2C1->TRISE = 43;

I2C1->CR1 |= I2C_CR1_PE; // Enable I2C Peripheral
}

void I2C1_Start(void)
{
    I2C1->CR1 |= I2C_CR1_START;
    while (!(I2C1->SR1 & I2C_SR1_SB))
        ; // Tunggu Start Bit terkirim
}

void I2C1_Stop(void)
{
    I2C1->CR1 |= I2C_CR1_STOP;
}

void I2C1_Addr(uint8_t addr)
{
    I2C1->DR = addr;
    while (!(I2C1->SR1 & I2C_SR1_ADDR))
        ; // Tunggu Address match
    (void)I2C1->SR2; // Baca SR2 untuk clear flag
}

void I2C1_Write(uint8_t data)
{
    I2C1->DR = data;
    while (!(I2C1->SR1 & I2C_SR1_BTF))
        ; // Tunggu Byte Transfer Finished
}

/* DRIVER PCF8574 LCD */
void LCD_Write_Nibble(uint8_t data, uint8_t control)
{
    uint8_t buf = (data & 0xF0) | control | 0x08; // 0x08 untuk Backlight ON
    I2C1_Start();
    I2C1_Addr(LCD_ADDR << 1); // Shift left 1 bit for Write
    I2C1_Write(buf); // En = 0
    I2C1_Write(buf | 0x04); // En = 1 (Pulse)
    I2C1_Write(buf); // En = 0
    I2C1_Stop();
}

```

```

    delay_ms(2);
}

void LCD_SendCmd(uint8_t cmd)
{
    LCD_Write_Nibble(cmd & 0xF0, 0);    // Upper nibble
    LCD_Write_Nibble((cmd << 4) & 0xF0, 0); // Lower nibble
}

void LCD_SendData(uint8_t data)
{
    LCD_Write_Nibble(data & 0xF0, 1);    // Upper nibble (RS=1)
    LCD_Write_Nibble((data << 4) & 0xF0, 1); // Lower nibble (RS=1)
}

void LCD_Init(void)
{
    // Inisialisasi 4-bit mode
    delay_ms(50);
    LCD_Write_Nibble(0x30, 0);
    delay_ms(5);
    LCD_Write_Nibble(0x30, 0);
    delay_ms(1);
    LCD_Write_Nibble(0x30, 0);
    delay_ms(1);
    LCD_Write_Nibble(0x20, 0);
    delay_ms(1); // Masuk 4-bit mode

    LCD_SendCmd(0x28); // Function Set: 4-bit, 2 Line, 5x8 Dots
    LCD_SendCmd(0x08); // Display OFF
    LCD_SendCmd(0x01); // Clear Display
    delay_ms(5);
    LCD_SendCmd(0x06); // Entry Mode
    LCD_SendCmd(0x0C); // Display ON, Cursor OFF
}

void LCD_String(char *str)
{
    while (*str)
        LCD_SendData(*str++);
}

void LCD_SetCursor(uint8_t row, uint8_t col)
{

```

```

uint8_t addr = (row == 0) ? 0x80 : 0xC0;
LCD_SendCmd(addr | col);
}

void LCD_Clear(void)
{
    LCD_SendCmd(0x01);
    delay_ms(2);
}

/* fungsi lain*/
void GPIO_Init(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIOCEN;

    // PA0 (Analog), PA1 (Alt Func TIM2), PC13 (Output)
    GPIOA->MODER |= (3 << (0 * 2));
    GPIOA->MODER &= ~(3 << (1 * 2));
    GPIOA->MODER |= (2 << (1 * 2));
    GPIOA->AFR[0] |= (1 << (1 * 4));

    GPIOC->MODER &= ~(3 << (13 * 2));
    GPIOC->MODER |= (1 << (13 * 2));
}

void ADC_Init(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_ADC1EN;
    ADC1->CR1 = 0;
    ADC1->SQR3 = 0;
    ADC1->CR2 |= ADC_CR2_ADON;
}

void TIM2_Init(void)
{
    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
    TIM2->PSC = 83;
    TIM2->ARR = 19999;
    TIM2->CCMR1 |= (6 << 12) | (1 << 11);
    TIM2->CCER |= (1 << 4);
    TIM2->CR1 |= TIM_CR1_ARPE | TIM_CR1_CEN;
}

void SystemClock_Config(void)

```

```

{
    RCC->CR |= RCC_CR_HSEON;
    while (!(RCC->CR & RCC_CR_HSERDY))
        ;
    RCC->APB1ENR |= RCC_APB1ENR_PWREN;
    PWR->CR |= PWR_CR_VOS;
    RCC->PLLCFGR = (25 << 0) | (336 << 6) | (1 << 16) | (4 << 24) | (1 << 22);
    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY))
        ;
    FLASH->ACR = FLASH_ACR_ICEN | FLASH_ACR_DCEN |
    FLASH_ACR_PRFTEN | FLASH_ACR_LATENCY_2WS;
    RCC->CFGR |= RCC_CFGR_PPRE1_DIV2;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL)
        ;
}

void delay_ms(uint32_t ms)
{
    for (uint32_t i = 0; i < ms * 4000; i++)
        __NOP();
}

```

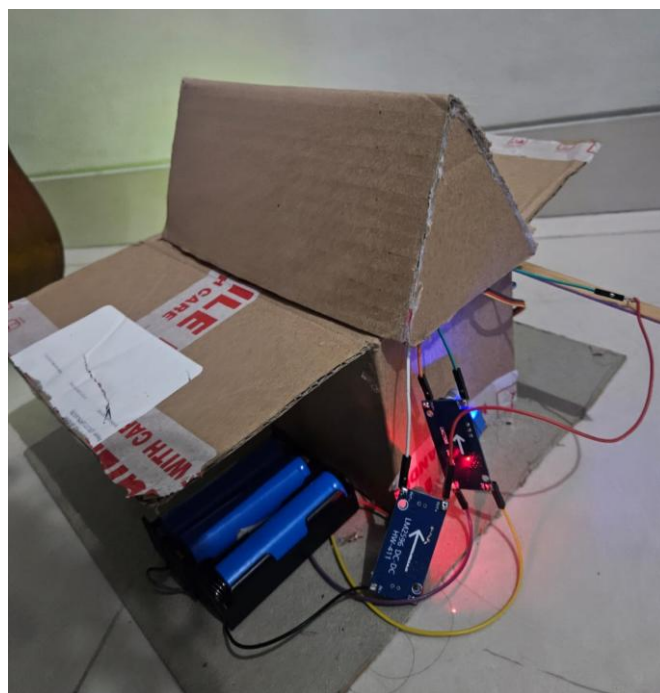
3.2 Demonstrasi Alat

https://drive.google.com/drive/folders/1Yl211-maFR0_PSEYa3JBHOwdXHsCVPgN?usp=drive_link

3.3 Realisasi Alat



Gambar 5 Tampak Depan



Gambar 6 Tampak Belakang



Gambar 7 Tampak Samping



Gambar 8 Tampak Dalam



Gambar 9 Status 1



Gambar 10 Status 2

BAB IV KESIMPULAN DAN SARAN

6.1 Kesimpulan

Berdasarkan hasil pengujian dan analisis, implementasi sistem secara keseluruhan telah berjalan sesuai dengan fungsi yang ditargetkan. Pada sisi input, konfigurasi register ADC1 terbukti mampu membaca data analog dari sensor FC-37 dengan presisi tinggi (FR-01), sehingga sistem dapat membedakan kondisi lingkungan kering dan hujan secara akurat. Data hasil pembacaan tersebut berhasil diintegrasikan dengan subsistem aktuator melalui kontrol PWM (FR-02), di mana pengaturan manual pada register TIM2 sukses menghasilkan sinyal 50Hz yang stabil. Stabilitas sinyal ini memungkinkan motor servo bergerak responsif ke posisi 90° saat hujan dan kembali ke 0° saat kering tanpa mengalami gangguan atau getaran (*jitter*). Selanjutnya, sebagai antarmuka visual bagi pengguna, implementasi protokol komunikasi I2C berbasis register berfungsi dengan baik (FR-04), memastikan status operasional sistem ("HUJAN!" atau "AMAN") dapat ditampilkan pada layar LCD 16x2 secara *real-time*.

6.2 Saran

1. Implementasikan fitur Sleep Mode pada STM32 untuk menghemat konsumsi baterai saat tidak hujan.
2. Tambahkan modul WiFi untuk mengirim notifikasi status hujan ke smartphone secara *real-time*.

DAFTAR PUSTAKA

- [1] STMicroelectronics, "STM32F401xB STM32F401xC Datasheet," Aug. 2015. [Online]. [Accessed: Jan. 3, 2026].
- [2] W. Song, W. Liu and Y. Pan, "Design of Intelligent Rainwater Detection Window Based on STM32 Single-Chip Microcomputer," 2019 Chinese Automation Congress (CAC), Hangzhou, China, 2019, pp. 278-281, doi: 10.1109/CAC48633.2019.8996685.
- [3] N. Sanap, T. Godbole, S. Shinde, and P. G. Gawande, "Automatic Rain Sensing Car Wiper Using 8052 Microcontroller," *International Journal for Multidisciplinary Research (IJFMR)*, vol. 7, no. 3, pp. 1–10, May–Jun. 2025